

Reglas YARA regulares e integradas con SSDEEP para el análisis de malware

Integrante 1

Integrante 2

Integrante 3

BETEK · Bootcamp de Ciberseguridad · Medellín, 2026

Table of Contents

Introducción

El presente laboratorio aborda la detección y clasificación de software malicioso mediante dos técnicas complementarias: las reglas YARA, orientadas a la identificación de patrones, y el algoritmo de hash difuso SSDEEP, orientado a la medición de similitud entre archivos. El trabajo se realiza porque, en el contexto de la seguridad informática, los atacantes modifican continuamente sus muestras para evadir la detección basada en firmas exactas, por lo que resulta necesario combinar enfoques de detección por patrón y por similitud. El laboratorio está pensado en dos fases: una de investigación, en la que se documentan los fundamentos de las reglas YARA, del algoritmo SSDEEP y de su integración en Python; y una de procedimiento, en la que se aplica lo investigado sobre una muestra real de malware en un entorno controlado (Kali Linux sobre máquina virtual). El método empleado consiste en instalar las herramientas, descargar una muestra real desde MalwareBazaar, escribir y ejecutar reglas YARA, generar y comparar hashes SSDEEP, e integrar ambas técnicas en un script de Python para evidenciar la efectividad de la detección combinada frente a la detección únicamente por reglas.

1. Investigación



1.1 Reglas YARA regulares

YARA es una herramienta enfocada, aunque no limitada, a la identificación y clasificación de muestras de malware, basada en patrones textuales o binarios. Cada regla YARA comienza con la palabra clave rule seguida de un identificador para la regla. Estos identificadores siguen las mismas convenciones que C y no pueden ser ninguna de las siguientes palabras reservadas:

- all
- and
- any
- ascii
- at
- base64
- base64wide
- condition
- contains
- endswith
- entrypoint
- false
- filesize
- for
- fullword
- global

import
icontains
iendswith
iequals
in
include
int16
int16be
int32
int32be
int8
int8be
startswith
matches
meta
nocase
none
not
of
or
private
rule
startswith
strings
them
true
uint16
uint16be

uint32
uint32be
uint8
uint8be
wide
xor
defined

Las reglas YARA se componen de dos secciones principales:

strings: puede ser omitida; en ella se delimitan los strings que serán parte de la regla. Cada uno consiste en un signo \$ seguido de un identificador, al cual se le asigna un valor (que puede ser hexadecimal, texto o expresión regular), y que puede ser usado en la condición de la regla.

condition: es obligatoria; es donde se definen las reglas lógicas y, por lo tanto, contiene expresiones booleanas que determinan si un archivo dado cumple con la regla definida.

Strings hexadecimales

Este tipo de strings son fundamentales para la definición de reglas YARA, puesto que permiten la combinación de comodines (*wild-cards*), operador de negación, rangos de bytes (*jumps*) y alternativas (*|*), que permiten definir patrones de bytes utilizables para identificar muestras de malware.

Strings de texto

Los strings de texto pueden usarse con una serie de modificadores que alteran la forma en la que serán interpretados. Estos modificadores se agregan al final del string separados por un espacio. Los modificadores de string de YARA se resumen en la siguiente tabla:

Keyword

Tipos

Resumen

Restricciones

nocase

Text, Regex

Ignora mayúsculas/minúsculas

No con xor, base64, base64wide

wide

Text, Regex

Emula UTF-16 intercalando bytes nulos (0x00)

Ninguna

ascii

Text, Regex

También busca en ASCII (necesario si se usa wide)

Ninguna

xor

Text

Aplica XOR de un byte al string

No con nocase, base64, base64wide

base64

Text

Genera 3 variantes codificadas en base64

No con nocase, xor, fullword

base64wide

Text

Como base64, luego intercala nulos como wide

No con nocase, xor, fullword

fullword

Text, Regex

La coincidencia no va precedida/seguida de alfanumérico

No con base64, base64wide

private

Hex, Text, Regex

La coincidencia nunca se incluye en la salida

Ninguna

Strings sensibles a mayúsculas

Los strings de YARA son *case sensitive* por defecto, pero se puede usar el modificador `nocase` para que la comparación sea insensible a mayúsculas y minúsculas. Por ejemplo:

```
$nocase_string = "test" nocase
```

de modo que `test`, `Test`, `TEST`, `tEsT`, etc., coinciden con esta regla.

Strings Unicode

Para definir un string Unicode se usa el modificador `wide`, para que YARA lo interprete como una secuencia de caracteres Unicode. Por ejemplo:

```
$unicode_string = "test" wide
```

aunque realmente solo intercala los códigos ASCII de los caracteres con un byte nulo (0x00). Si se desea buscar en modo ASCII y `wide` a la vez, se usa `ascii` junto con `wide`:

```
$unicode_string = "test" ascii wide
```

Strings XOR

Es útil para buscar patrones de bytes que han sido ofuscados mediante un operador XOR con un byte clave. Además, se puede combinar con los modificadores `wide` y `ascii`, y también admite un rango de claves. Por ejemplo:

```
$xor_string = "This program cannot" xor(0x01-0xFF)
```

Strings Base64

Se usa el modificador `base64` para buscar texto que ha sido codificado en base64. El modificador `base64wide` funciona de manera similar, pero el resultado se convierte a una secuencia de bytes en formato Unicode. Se relacionan con `wide` y `ascii` de forma tal que estos se aplican antes de la codificación en base64. Este modificador no funciona con hexadecimal.

Expresiones regulares

Del mismo modo que los strings hexadecimales, las expresiones regulares son fundamentales para la definición de reglas YARA, ya que permiten definir patrones de texto utilizables para identificar muestras de malware. Estas expresiones se delimitan con el carácter `/`. Las expresiones regulares de YARA son compatibles con los modificadores `nocase`, `wide` y `ascii`, así como con los caracteres `i` o `s` para especificar, respectivamente, si son insensibles a mayúsculas o si el punto (.) coincide con saltos de línea.

Condiciones

Las condiciones en YARA se reducen a expresiones booleanas, que pueden contener operadores lógicos, relacionales, aritméticos y a nivel de bits. Para los operadores a nivel de byte es importante tener en cuenta que los enteros son siempre de 64 bits. Estos operadores siguen

el mismo orden de precedencia y la misma regla de asociatividad que en C, con la diferencia de que los operadores relacionales están por debajo, en el orden de precedencia, de los operadores a nivel de bits.

Conteo de strings

El número de coincidencias de un string en particular puede contarse usando el operador `#` seguido del identificador del string. Por ejemplo, si tenemos un string definido como:

```
$my_string = "test"
```

podemos contar el número de coincidencias en un archivo con la siguiente condición:

```
#my_string > 5
```

Esto significa que la regla se cumple si el string test aparece más de 5 veces.

Tamaño de archivo

Se pueden establecer condiciones basadas en el tamaño del archivo usando la variable especial `filesize` combinada con operadores relacionales. Además, se pueden usar postfijos como KB, MB y GB. Por ejemplo, la siguiente condición se cumple si el tamaño del archivo es mayor a 1 MB:

```
filesize > 1MB
```

Conjuntos de strings (sets of strings)

Cuando una regla maneja varios strings, en lugar de combinarlos uno por uno con `and/or`, YARA permite tratarlos como un conjunto y preguntar cuántos coinciden mediante el operador `of`. La forma general es `<cantidad> of <conjunto>`, donde la cantidad puede ser un número o las palabras clave `any` (equivale a 1 `of`), `all` o `none`; y el conjunto puede ser una lista explícita, un comodín (`$str*`) o `them` (todos los strings de la regla). Por ejemplo:

```
any of them      // al menos uno de los definidos
all of them      // todos
2 of ($a, $b, $c) // al menos 2 de esos 3
none of ($b*)    // ninguno de los que empiezan por $b
```

Bucles (loops)

El operador `of` es en realidad un caso particular de una construcción más general: el bucle `for`. Un bucle aplica una misma condición a cada

elemento de una colección y comprueba para cuántos elementos se cumple. Su forma general es:

```
for <cantidad> <variable> in <colección> : ( <condición> )
```

A diferencia de `of`, el bucle permite evaluar una condición distinta sobre cada string. Dentro del bucle, `$` representa el string actual. Por ejemplo, la siguiente condición se cumple si alguno de los strings aparece al inicio del archivo (offset 0):

```
for any of ($a, $b, $c) : ( $ at 0 )
```

Iteradores (iterators)

El iterador es la colección concreta que recorre el bucle. Partiendo de lo más simple, YARA permite iterar, entre otros, sobre rangos o listas de enteros ((1..3) o (1, 2, 3)) y sobre las ocurrencias de un string. Para esto último se usan los operadores `#a` (número de coincidencias del string `$a`), `@a[i]` (offset de la *i*-ésima coincidencia) y `!a[i]` (longitud de esa coincidencia), lo que permite razonar sobre la posición exacta de cada coincidencia. Por ejemplo, la siguiente condición exige que todas las ocurrencias de `$a` estén en los primeros 100 bytes del archivo:

```
for all i in (1..#a) : ( @a[i] < 100 )
```

1.2 Algoritmos SSDEEP

¿Qué es SSDEEP?

SSDEEP es una herramienta que utiliza una técnica conocida como *fuzzy hashing* o hash difuso. A diferencia de un hash tradicional como MD5 o SHA256, que cambia completamente si se modifica un solo byte del archivo, SSDEEP permite encontrar similitudes entre archivos. Esto es útil en ciberseguridad porque muchas veces los atacantes modifican un malware levemente para evitar su detección: aunque el archivo cambie, puede seguir siendo muy parecido al original, y SSDEEP ayuda a identificar que ambos archivos tienen una relación o parecido. En otras palabras, SSDEEP no busca saber si dos archivos son idénticos, sino qué tan parecidos son.

¿Cómo funciona de forma general?

El funcionamiento de SSDEEP se basa en dividir el contenido del archivo en bloques y generar una firma basada en esos fragmentos. Luego, al comparar dos firmas, se obtiene un porcentaje de similitud:

Si dos archivos son exactamente iguales, la similitud será muy alta o prácticamente total.

Si dos archivos son diferentes pero comparten gran parte de su estructura o contenido, la similitud será intermedia.

Si no tienen relación, la similitud será muy baja o cero.

Esto hace que SSDEEP sea muy útil para identificar variantes de un mismo malware, comparar muestras sospechosas, agrupar archivos similares y detectar modificaciones menores en familias de malware.

Diferencia entre hash tradicional y hash difuso

Los **hashes tradicionales** (MD5, SHA1, SHA256) identifican un archivo de forma exacta: si el archivo cambia, aunque sea muy poco, el hash cambia por completo. Sirven para verificar integridad, pero no para medir similitud. El **hash difuso** (SSDEEP) permite comparar archivos parecidos, no necesita que sean idénticos y entrega un porcentaje de similitud. Por ejemplo, si un archivo malicioso original es modificado por un atacante cambiando solo unas líneas o agregando pequeños datos, su SHA256 será totalmente diferente; sin embargo, con SSDEEP todavía se puede detectar que existe una gran similitud entre ambos archivos.

Importancia de SSDEEP en el análisis de malware

SSDEEP es importante porque los atacantes suelen reutilizar partes de código, empaquetar malware, cambiar nombres o alterar pequeñas secciones del archivo para evitar la detección basada en firmas exactas. En este contexto, SSDEEP ayuda a detectar variantes del mismo malware, encontrar relaciones entre muestras, apoyar la clasificación y reducir el tiempo de análisis cuando existen muchas muestras parecidas. Por ejemplo, si un analista ya conoce una muestra y luego recibe otra sospechosa, puede calcular el SSDEEP de ambas y ver si existe un nivel alto de similitud; si el porcentaje es alto, puede indicar que pertenecen a la misma familia o que una es variante de la otra.

Ventajas y limitaciones

Entre las **ventajas** de SSDEEP se encuentran: permite encontrar similitud (no solo archivos idénticos, sino también relacionados), es útil para variantes de malware modificadas ligeramente, apoya el análisis forense y la clasificación, y es relativamente fácil de implementar mediante herramientas y librerías en Python.

Entre sus **limitaciones**: no reemplaza otros métodos de análisis (es un apoyo, no sustituye a YARA, ni al análisis estático o dinámico); puede

generar falsos positivos; puede fallar si los cambios son muy grandes o si el malware fue muy ofuscado o reempaquetado; y depende del contexto, ya que un porcentaje de similitud por sí solo no siempre basta para concluir que dos archivos son de la misma familia. Por eso se recomienda combinar SSDEEP con otras técnicas, como las reglas YARA.

Ejemplo conceptual de uso

Supongamos que un analista tiene una muestra conocida de malware y genera su firma SSDEEP; después recibe un archivo sospechoso nuevo. El proceso sería: generar el hash difuso del malware conocido, generar el hash difuso del archivo sospechoso, comparar ambos hashes y revisar el porcentaje de similitud. Si el resultado es, por ejemplo, 85 %, se puede considerar que hay una relación fuerte entre ambas muestras; si es muy bajo (5 % o 10 %), probablemente no exista relación significativa. El porcentaje no es una prueba absoluta, sino un indicador que debe analizarse junto con otros datos.

1.3 Reglas YARA integradas con SSDEEP en código Python

¿Qué significa integrar YARA con SSDEEP?

YARA y SSDEEP cumplen funciones diferentes, pero se complementan muy bien: YARA sirve para detectar patrones específicos dentro de un archivo, y SSDEEP para medir similitud entre archivos. Integrar ambos en Python significa desarrollar un programa que use las dos técnicas al mismo tiempo para mejorar la identificación de muestras sospechosas. Por ejemplo, un script en Python puede cargar una regla YARA, revisar si el archivo coincide con esa regla, calcular el SSDEEP del archivo, compararlo con una muestra conocida y generar un resultado más completo. Esto es útil porque una regla YARA puede detectar características muy puntuales, mientras que SSDEEP ayuda a identificar parecidos generales.

¿Por qué usar Python?

Python es un lenguaje muy utilizado en ciberseguridad porque tiene una sintaxis sencilla, permite automatizar tareas, cuenta con librerías para análisis de archivos y se integra fácilmente con herramientas como YARA y SSDEEP. Además, en entornos como Kali Linux es común usar Python para crear scripts de laboratorio.

Lógica básica de integración

La integración entre YARA y SSDEEP en Python normalmente sigue esta lógica:

Definir o cargar una regla YARA, que busca cadenas, patrones o condiciones concretas dentro del archivo.

Seleccionar una muestra de referencia, es decir, un archivo conocido (por ejemplo, una muestra confirmada de malware).

Calcular el hash SSDEEP del archivo de referencia y del archivo a analizar.

Comparar la similitud con SSDEEP para obtener un porcentaje.

Tomar una decisión, evaluando si la regla YARA detecta coincidencia y si el porcentaje de similitud supera un umbral definido. Si ambas condiciones se cumplen, hay una mayor probabilidad de que el archivo esté relacionado con la amenaza buscada.

Beneficio de combinar ambas técnicas

Combinar YARA con SSDEEP tiene sentido porque ambas herramientas se complementan. YARA aporta precisión en la búsqueda de patrones concretos, detección por firmas o cadenas y reglas específicas para familias de malware. SSDEEP aporta comparación por similitud, detección de variantes y análisis de archivos modificados. El resultado de la integración es una detección más robusta, mejor apoyo al analista y mayor contexto sobre el archivo sospechoso. En otras palabras, YARA responde mejor a la pregunta «¿este archivo contiene patrones que ya conozco?», mientras que SSDEEP responde mejor a «¿este archivo se parece a otro malware conocido?».

Ejemplo de flujo en Python

A continuación se presenta un ejemplo sencillo, de uso educativo, que muestra la lógica general de la integración:

```
import yara
import ssdeep
```

```
# Compilar regla YARA
```

```
rules = yara.compile(filepath='regla_malware.yar')
```

```
# Archivo sospechoso y muestra de referencia
```

```
archivo_sospechoso = 'muestra_sospechosa.bin'
```

```
archivo_referencia = 'malware_referencia.bin'

# Ejecutar regla YARA
matches = rules.match(archivo_sospechoso)

# Calcular hashes SSDEEP
hash_sospechoso = ssdeep.hash_from_file(archivo_sospechoso)
hash_referencia = ssdeep.hash_from_file(archivo_referencia)

# Comparar similitud
similitud = ssdeep.compare(hash_sospechoso, hash_referencia)

print("Coincidencias YARA:", matches)
print("Hash SSDEEP sospechoso:", hash_sospechoso)
print("Hash SSDEEP referencia:", hash_referencia)
print("Porcentaje de similitud:", similitud)

# Ejemplo de decisión
if matches and similitud >= 70:
    print("El archivo presenta coincidencia YARA y alta similitud con la referencia.")
elif matches:
    print("El archivo coincide con la regla YARA, pero la similitud SSDEEP no es alta.")
elif similitud >= 70:
    print("El archivo no coincide con la regla YARA, pero se parece bastante a la referencia.")
else:
    print("No se encontraron evidencias fuertes de relación.")
```

Este ejemplo importa las librerías yara y ssdeep, carga una regla YARA, analiza un archivo sospechoso, calcula el hash difuso del archivo sospechoso y de una muestra conocida, compara ambos hashes, muestra el porcentaje de similitud y, finalmente, toma una decisión básica con base en ambos resultados.

Ejemplo básico de una regla YARA

Para complementar el ejemplo anterior, una regla YARA sencilla podría verse así:

```
rule Ejemplo_Malware
{
    meta:
```

```
description = "Regla simple de ejemplo para detectar cadenas sospechosas"
```

```
author = "Estudiante"
```

```
date = "2026-06-18"
```

```
strings:
```

```
$a = "powershell" nocase
```

```
$b = "cmd.exe" nocase
```

```
$c = "http://" nocase
```

```
$d = "https://" nocase
```

```
condition:
```

```
2 of them
```

```
}
```

Esta regla busca ciertas cadenas comunes que podrían aparecer en archivos o scripts maliciosos; si encuentra al menos dos, genera coincidencia. En un entorno real, las reglas deben diseñarse con más cuidado para evitar falsos positivos.

Interpretación de resultados

Al integrar YARA y SSDEEP, los resultados pueden interpretarse en cuatro casos:

YARA coincide y SSDEEP alto: es el caso más fuerte; el archivo contiene patrones conocidos y se parece mucho a una muestra maliciosa de referencia.

YARA coincide y SSDEEP bajo: el archivo comparte ciertos patrones, pero no es muy parecido a la referencia; puede ser otra variante o un falso positivo.

YARA no coincide y SSDEEP alto: puede ocurrir si el archivo fue modificado y ya no cumple exactamente la regla, pero conserva mucha similitud estructural con la muestra conocida.

YARA no coincide y SSDEEP bajo: hay poca evidencia de relación con la amenaza analizada.

Por eso, la combinación de ambas técnicas ofrece una visión más completa que usar solo una.

Ventajas y limitaciones de la integración

Las **ventajas** de integrar YARA con SSDEEP en Python son: la automatización del análisis, la combinación de detección exacta y por

similitud, mejor apoyo para identificar variantes, la posibilidad de procesar múltiples muestras y la facilidad de adaptación a laboratorios de investigación. Las **limitaciones** son: depende de la calidad de las reglas YARA y de contar con muestras de referencia adecuadas, SSDEEP no siempre detecta similitud si el malware cambia demasiado, el script puede requerir ajustes según el tipo de archivo y no reemplaza un análisis manual o más profundo.

2. Procedimiento de laboratorio

Instalacion de YARA

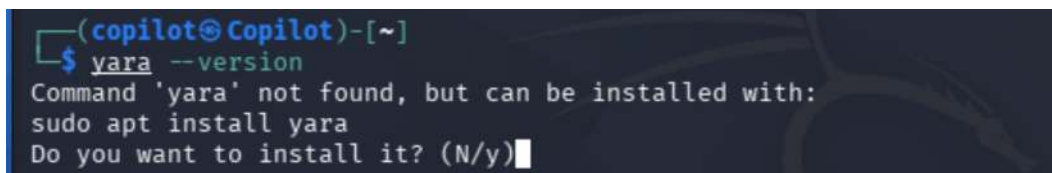
YARA puede ser instalada en nuestro sistema operativo (linux en este caso), siguiendo el procedimiento en:

<https://yara.readthedocs.io/en/latest/gettingstarted.html>

Para obtener la ultima version de YARA y poder usar módulos adicionales. Por otro lado podemos instalar YARA desde los repositorios de nuestra distribucion linux, con el siguiente comando:

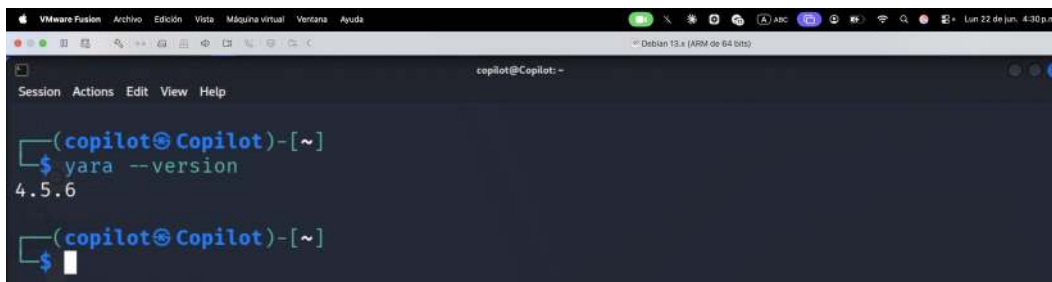
```
sudo apt install yara
```

la desventaja es que no nos entrega necesariamente la ultima version, pero para los propósitos de este laboratorio introductorio es suficiente, ya que no necesitamos de módulos adicionales. Primero observamos que YARA no esta instalada en nuestro sistema linux,



```
(copilot@Copilot)-[~]
$ yara --version
Command 'yara' not found, but can be installed with:
sudo apt install yara
Do you want to install it? (N/y)
```

como se observa en la imagen, al ejecutar el comando yara, nos indica que no se encuentra instalado. Por lo que procedemos a instalarlo:



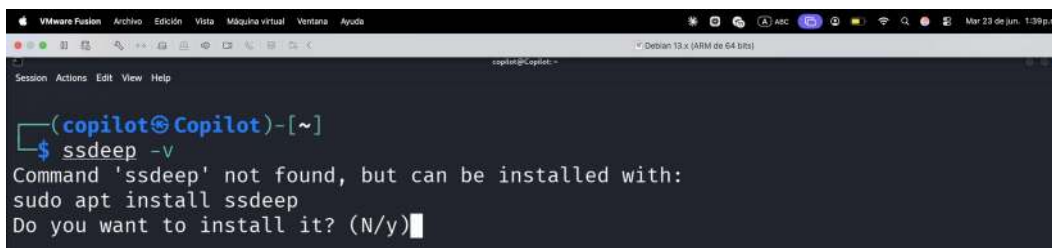
```
(copilot@Copilot)-[~]
$ yara --version
4.5.6
(copilot@Copilot)-[~]
$
```

Instalación SSDEEP

Primero verificamos que si SSDEEP se encuentra instalado en nuestro sistema linux, ejecutando el comando:

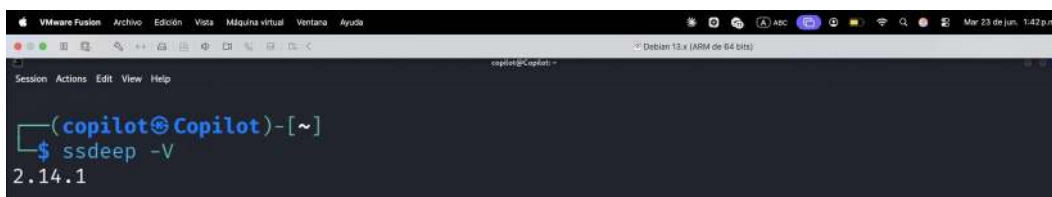
```
ssdeep -Vcl
```

Como observamos en la imagen, nos indica que no se encuentra instalado, por lo que procedemos a instalarlo con la instalación automática de paquetes de nuestra distribución como se recomienda en: <https://ssdeep-project.github.io/ssdeep/usage.html#install>



```
(copilot@Copilot)-[~]  
$ ssdeep -v  
Command 'ssdeep' not found, but can be installed with:  
sudo apt install ssdeep  
Do you want to install it? (N/y)
```

y verificamos con el mismo comando anterior que se encuentra instalado correctamente:



```
(copilot@Copilot)-[~]  
$ ssdeep -V  
2.14.1
```

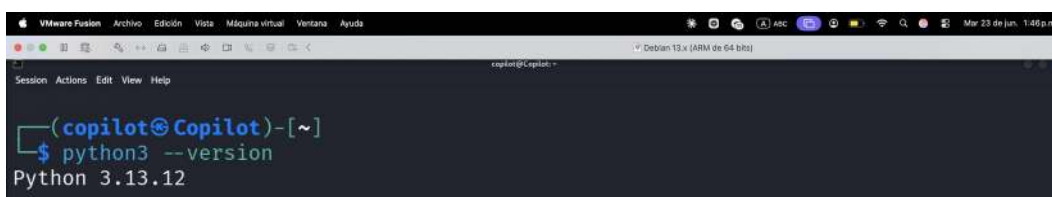
Instalación python-ssdeep

Verificamos que python se encuentra instalado en nuestro sistema linux, ejecutando el comando:

```
python3 --version
```

Como vemos en la imagen, nos indica que python3 se encuentra instalado, por lo que procedemos a instalar el modulo python-ssdeep con el siguiente comando:

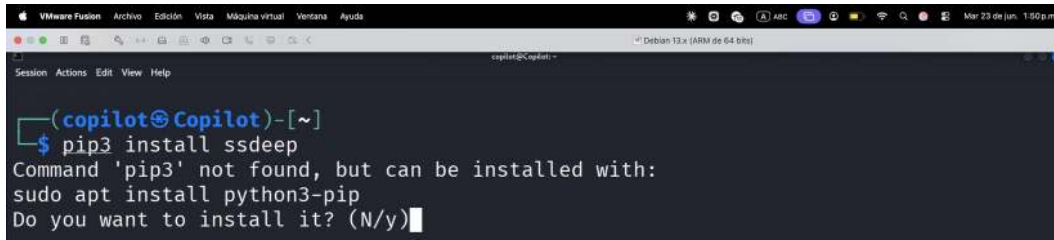
```
pip3 install yara-python pydeep
```



```
(copilot@Copilot)-[~]  
$ python3 --version  
Python 3.13.12
```

Observamos en la siguiente imagen que paquete asociado con pip3 no se encuentra instalado, por lo que procedemos a instalarlo confirmado el

mensaje de instalación:

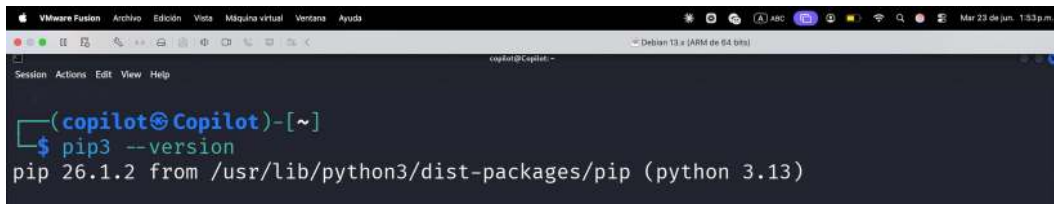


```
(copilot@Copilot)-[~]
$ pip3 install ssdeep
Command 'pip3' not found, but can be installed with:
sudo apt install python3-pip
Do you want to install it? (N/y)
```

y verificamos la correcta instalación del pip3 con el siguiente comando:

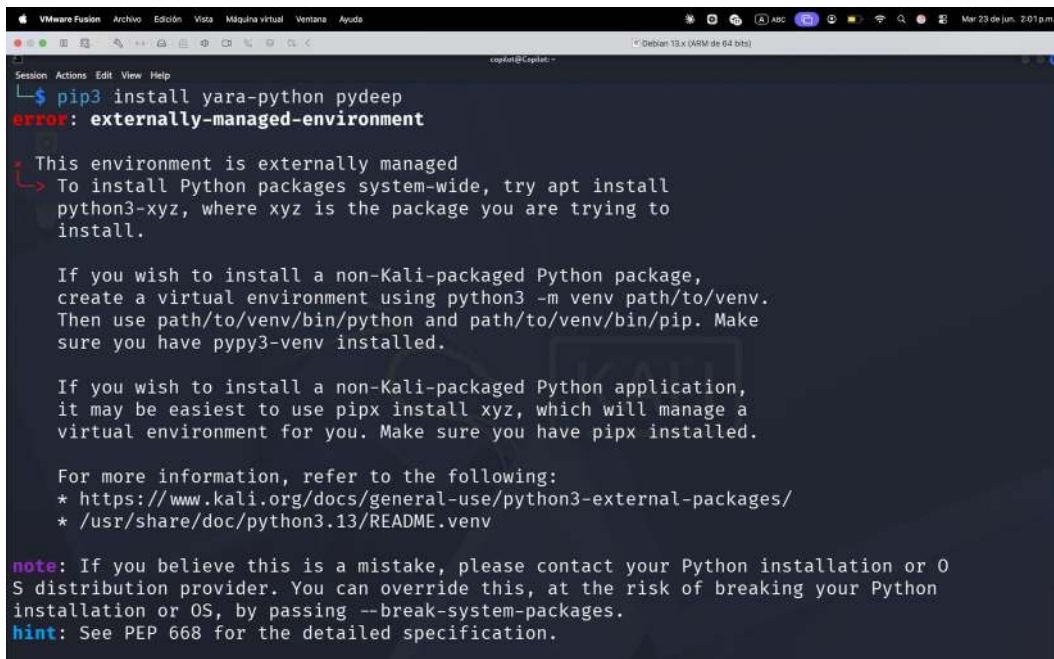
`pip3 --version`

Observamos que nos indica la versión de pip3 instalada en nuestro sistema linux, por lo que procedemos a instalar el modulo python-ssdeep.



```
(copilot@Copilot)-[~]
$ pip3 --version
pip 26.1.2 from /usr/lib/python3/dist-packages/pip (python 3.13)
```

Pero observamos el siguiente error a la hora de ejecutar el comando para la instalación de python-ssdeep:



```
(copilot@Copilot)-[~]
$ pip3 install yara-python pydeep
error: externally-managed-environment

This environment is externally managed
To install Python packages system-wide, try apt install
python3-xyz, where xyz is the package you are trying to
install.

If you wish to install a non-Kali-packaged Python package,
create a virtual environment using python3 -m venv path/to/venv.
Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make
sure you have pypy3-venv installed.

If you wish to install a non-Kali-packaged Python application,
it may be easiest to use pipx install xyz, which will manage a
virtual environment for you. Make sure you have pipx installed.

For more information, refer to the following:
* https://www.kali.org/docs/general-use/python3-external-packages/
* /usr/share/doc/python3.13/README.venv

note: If you believe this is a mistake, please contact your Python installation or O
S distribution provider. You can override this, at the risk of breaking your Python
installation or OS, by passing --break-system-packages.
hint: See PEP 668 for the detailed specification.
```

Este mensaje es debido a que a partir de Kali Linux 2024.4, se desaconseja el uso de pip para instalar paquetes externos a nivel del sistema, pues las utilidades Kali Linux 2024.4 usan Python, y la instalación de paquetes externos a nivel del sistema puede causar conflictos.

[https://www.kali.org.translate.google/docs/general-use/python3-external-packages/?](https://www.kali.org.translate.google/docs/general-use/python3-external-packages/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc#introduction-say-good-bye-to-pip-install)

[_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc#introduction-say-good-bye-to-pip-install](https://www.kali.org.translate.google/docs/general-use/python3-external-packages/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc#introduction-say-good-bye-to-pip-install)

Pero como observamos en <https://pkg.kali.org/pkg/python-ssdeep>, el paquete python-ssdeep se encuentra disponible en los repositorios de Kali Linux, por lo que podemos instalarlo con el siguiente comando:

`sudo apt install python3-ssdeep`

y verificamos la correcta instalación del modulo python-ssdeep con el siguiente comando:

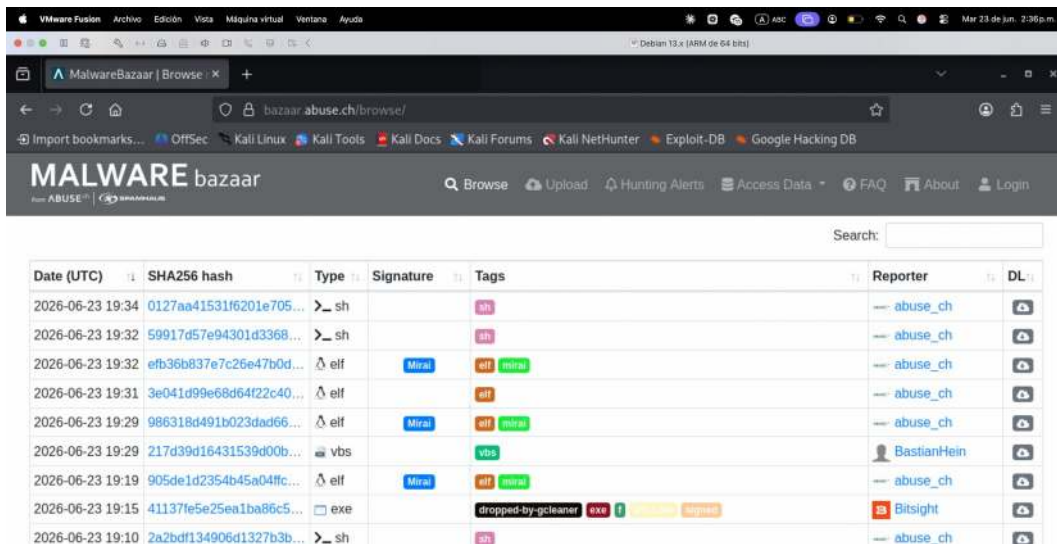
`python3 -c "import ssdeep; print(ssdeep.__version__)"`



```
(copilot@Copilot)-[~]
$ python3 -c "import ssdeep; print('ssdeep', ssdeep.__version__)"
ssdeep 3.4.1
```

Descarga de muestras de malware

Para esto entramos al sitio web **malware bazaar** (<https://bazaar.abuse.ch/>) y descargamos las muestras de malware que se nos indica en el laboratorio.

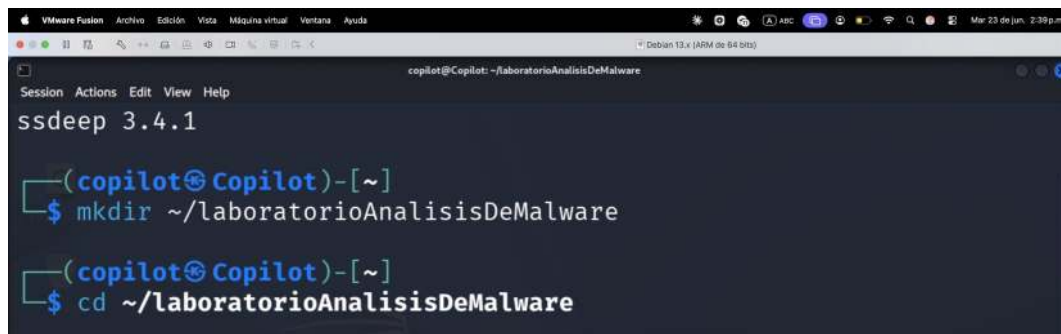


Date (UTC)	SHA256 hash	Type	Signature	Tags	Reporter	DL
2026-06-23 19:34	0127aa41531f6201e705...	sh		sh	abuse_ch	
2026-06-23 19:32	59917d57e94301d3368...	sh		sh	abuse_ch	
2026-06-23 19:32	efb36b837e7c26e47b0d...	elf	Mirai	elf mirai	abuse_ch	
2026-06-23 19:31	3e041d99e68d64f22c40...	elf		elf	abuse_ch	
2026-06-23 19:29	986318d491b023dad66...	elf	Mirai	elf mirai	abuse_ch	
2026-06-23 19:29	217d39d16431539d00b...	vbs		vbs	BastianHein	
2026-06-23 19:19	905de1d2354b45a04ffc...	elf	Mirai	elf mirai	abuse_ch	
2026-06-23 19:15	41137fe5e25ea1ba86c5...	exe		dropped-by-cleaner exe f	Bitsight	
2026-06-23 19:10	2a2bdf134906d1327b3b...	sh		sh	abuse_ch	

y creamos el entorno de trabajo para el laboratorio, con los siguientes comando:

`mkdir ~/laboratorioAnálisisDeMalware`

`cd ~/laboratorioAnálisisDeMalware`

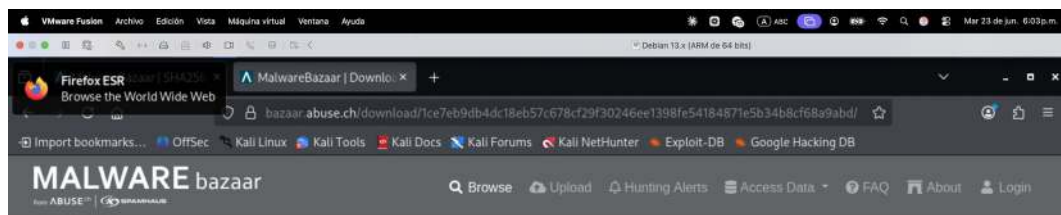


```
copilot@Copilot: ~/laboratorioAnálisisDeMalware
ssdeep 3.4.1

(copilot@Copilot)-[~]
$ mkdir ~/laboratorioAnálisisDeMalware

(copilot@Copilot)-[~]
$ cd ~/laboratorioAnálisisDeMalware
```

y buscamos filtrando por el tipo de sha256 de malware para el wannacry sha256:1ce7eb9db4dc18eb57c678cf29f30246ee1398fe54184871e5b34b8cf68a9abd y seleccionamos un ejecutable de tipo .exe, antes de descargarlo en el entorno, tomamos un snapshot de la maquina virtual para poder restaurarla en caso de que el malware se ejecute y cause daños en el sistema operativo y no aseguramos que la configuracion de la maquina virtual este en modo NAT.



Browse / Download

MalwareBazaar Database

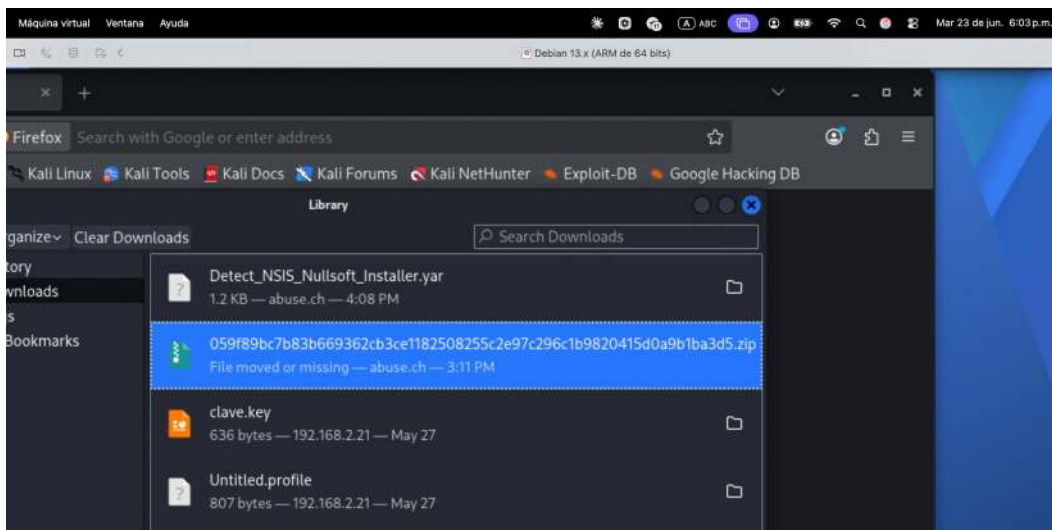
This page let you download the following malware sample: SHA256 1ce7eb9db4dc18eb57c678cf29f30246ee1398fe54184871e5b34b8cf68a9abd

Caution!

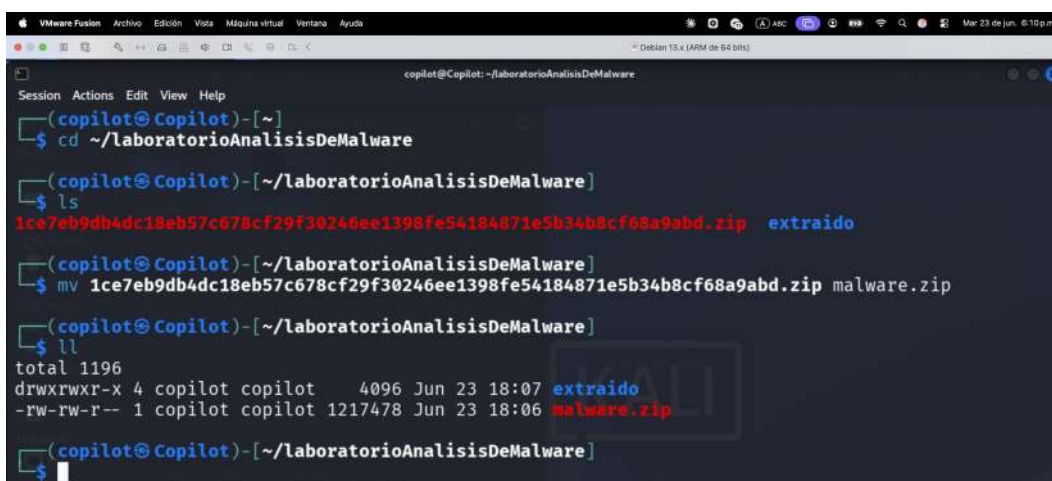
You are about to download a malware sample. By clicking on "download", you declare that you have understood what you are doing and that MalwareBazaar can not be held accountable for any damage caused by downloading this malware sample!

ZIP password: infected

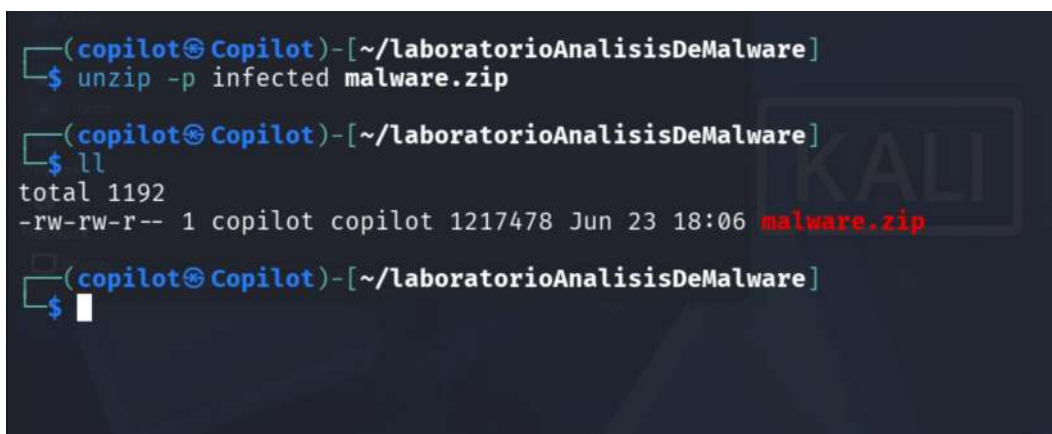
Download



Lo renombramos y lo movemos a la carpeta de laboratorio, verificamos ademas que el archivo se encuentra en la carpeta de laboratorio:



Y procedemos a extraer el archivo comprimido con el siguiente comando:
unzip malwareAgentTesla.zip



Y observamos que no fue posible extraer el archivo comprimido debido a

que este viene cifrado con AES y unzip no soporta, por lo que procedemos a instalar **7zip** con el siguiente comando:

```
sudo apt install p7zip-full
```

y realizamos la extracción del archivo comprimido con el siguiente comando:

```
7z x -pinfected malware.zip
```

y verificamos que el archivo comprimido fue extraído correctamente:

```
(copilot@Copilot)-[~/laboratorioAnálisisDeMalware]
$ ll
total 2528
-rw-r--r-- 1 copilot copilot 1366016 Jun 23 2026 malware.exe
-rw-rw-r-- 1 copilot copilot 1217478 Jun 23 18:06 malware.zip

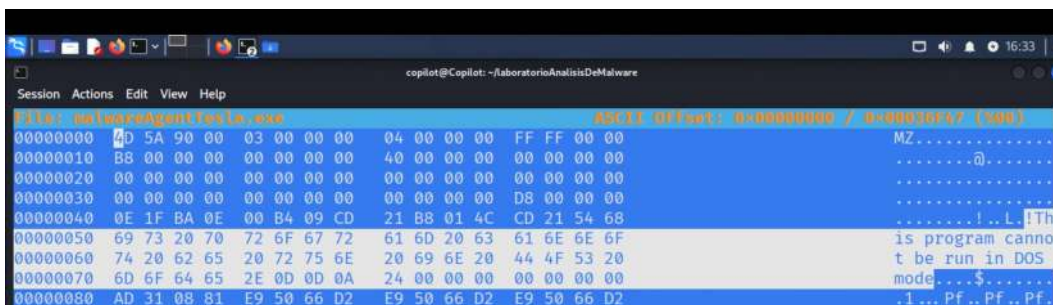
(copilot@Copilot)-[~/laboratorioAnálisisDeMalware]
$
```

y lo renombramos a **malwareAgentTesla.exe** para poder trabajar con el archivo ejecutable de malware mas fácilmente.

Creacion de reglas YARA

Creación de reglas YARA basica

Si analizamos en hexeditor el archivo ****malware.exe***, podemos observar que contiene los siguientes strings:



```
File: malwareAgentTesla.exe
00000000 4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 MZ.....
00000010 B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00 .....@.....
00000020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000030 00 00 00 00 00 00 00 00 00 00 00 00 D8 00 00 00 .....
00000040 0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68 .....!..L!Th
00000050 69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F is program cannot
00000060 74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20 t be run in DOS
00000070 6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00 mode....$.....
00000080 AD 31 08 81 E9 50 66 D2 E9 50 66 D2 E9 50 66 D2 .1... Pf.. Pf.. Pf..
```

This program cannot be run in DOS mode

Abrimos un editor de texto y creamos una regla YARA básica para el malware AgentTesla que descargamos de AntiDebugging.yara y la renombramos a **reglas_wannacry.yar** y la guardamos en la carpeta de laboratorio, con el siguiente contenido:

```
rule DebuggerCheck__API : AntiDebug DebuggerCheck {
    meta:
        weight = 1
    strings:
```

```
    $ = "IsDebuggerPresent"
condition:
    any of them
}
```

```
rule DebuggerCheck__PEB : AntiDebug DebuggerCheck {
    meta:
        weight = 1
    strings:
        $ = "IsDebugged"
    condition:
        any of them
}
```

```
rule DebuggerCheck__GlobalFlags : AntiDebug DebuggerCheck {
    meta:
        weight = 1
    strings:
        $ = "NtGlobalFlags"
    condition:
        any of them
}
```

```
rule DebuggerCheck__QueryInfo : AntiDebug DebuggerCheck {
    meta:
        weight = 1
    strings:
        $ = "QueryInformationProcess"
    condition:
        any of them
}
```

```
rule DebuggerCheck__RemoteAPI : AntiDebug DebuggerCheck {
    meta:
        weight = 1
    strings:
        $ = "CheckRemoteDebuggerPresent"
    condition:
        any of them
}
```

```
////////////////////////////////////  
rule DebuggerHiding__Thread : AntiDebug DebuggerHiding {  
  meta:  
    weight = 1  
  strings:  
    $ = "SetInformationThread"  
  condition:  
    any of them  
}
```

```
rule DebuggerHiding__Active : AntiDebug DebuggerHiding {  
  meta:  
    weight = 1  
  strings:  
    $ = "DebugActiveProcess"  
  condition:  
    any of them  
}
```

```
rule DebuggerTiming__PerformanceCounter : AntiDebug  
DebuggerTiming {  
  meta:  
    weight = 1  
  strings:  
    $ = "QueryPerformanceCounter"  
  condition:  
    any of them  
}
```

```
rule DebuggerTiming__Ticks : AntiDebug DebuggerTiming {  
  meta:  
    weight = 1  
  strings:  
    $ = "GetTickCount"  
  condition:  
    any of them  
}
```

```
rule DebuggerOutput__String : AntiDebug DebuggerOutput {  
  meta:  
    weight = 1
```

```
strings:
    $ = "OutputDebugString"
condition:
    any of them
}
```

```
////////////////////////////////////
rule DebuggerException__UnhandledFilter : AntiDebug
DebuggerException {
    meta:
        weight = 1
    strings:
        $ = "SetUnhandledExceptionFilter"
    condition:
        any of them
}
```

```
rule DebuggerException__ConsoleCtrl : AntiDebug DebuggerException {
    meta:
        weight = 1
    strings:
        $ = "GenerateConsoleCtrlEvent"
    condition:
        any of them
}
```

```
rule DebuggerException__SetConsoleCtrl : AntiDebug
DebuggerException {
    meta:
        weight = 1
    strings:
        $ = "SetConsoleCtrlHandler"
    condition:
        any of them
}
```

```
////////////////////////////////////
rule ThreadControl__Context : AntiDebug ThreadControl {
    meta:
        weight = 1
    strings:
```

```
    $ = "SetThreadContext"
condition:
    any of them
}
```

```
rule DebuggerCheck__DrWatson : AntiDebug DebuggerCheck {
    meta:
        weight = 1
    strings:
        $ = "__invoke__watson"
    condition:
        any of them
}
```

```
rule SEH__v3 : AntiDebug SEH {
    meta:
        weight = 1
    strings:
        $ = "___except__handler3"
        $ = "___local__unwind3"
    condition:
        any of them
}
```

```
rule SEH__v4 : AntiDebug SEH {
    // VS 8.0+
    meta:
        weight = 1
    strings:
        $ = "___except__handler4"
        $ = "___local__unwind4"
        $ = "__XcptFilter"
    condition:
        any of them
}
```

```
rule SEH__vba : AntiDebug SEH {
    meta:
        weight = 1
    strings:
        $ = "vbaExceptionHandler"
```

```
condition:
    any of them
}
```

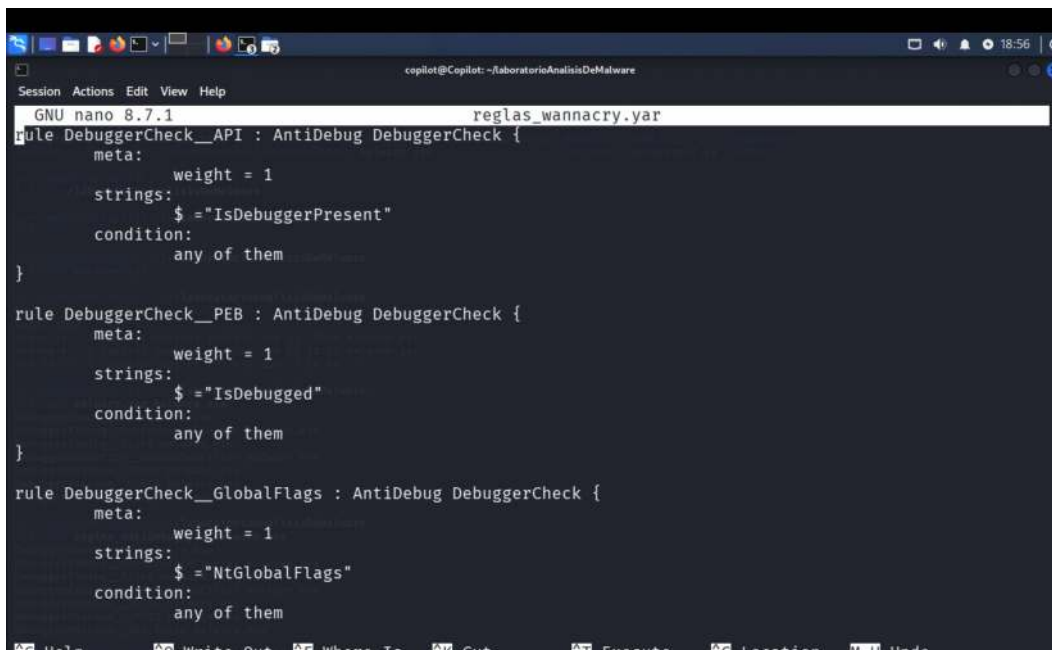
```
rule SEH__vectorized : AntiDebug SEH {
    meta:
        weight = 1
    strings:
        $ = "AddVectoredExceptionHandler"
        $ = "RemoveVectoredExceptionHandler"
    condition:
        any of them
}
```

```
//////////////////////////////// Patterns
rule DebuggerPattern__RDTSC : AntiDebug DebuggerPattern {
    meta:
        weight = 1
    strings:
        $ = {0F 31}
    condition:
        any of them
}
```

```
rule DebuggerPattern__CPUID : AntiDebug DebuggerPattern {
    meta:
        weight = 1
    strings:
        $ = {0F A2}
    condition:
        any of them
}
```

```
rule DebuggerPattern__SEH_Saves : AntiDebug DebuggerPattern {
    meta:
        weight = 1
    strings:
        $ = {64 ff 35 00 00 00 00}
    condition:
        any of them
}
```

```
rule DebuggerPattern__SEH_Inits : AntiDebug DebuggerPattern {  
  meta:  
    weight = 1  
  strings:  
    $a = { 64 A3 00 00 00 00 }  
    $b = { 64 89 25 00 00 00 00 }  
  condition:  
    $a or $b  
}
```



The screenshot shows a terminal window with the nano text editor open. The file being edited is named 'reglas_wannacry.yar'. The editor displays three YARA rules for detecting debuggers. The first rule, 'DebuggerPattern__API', checks for the 'IsDebuggerPresent' string. The second rule, 'DebuggerCheck__PEB', checks for the 'IsDebugged' string. The third rule, 'DebuggerCheck__GlobalFlags', checks for the 'NtGlobalFlags' string. Each rule has a meta-section with a weight of 1 and a condition that is 'any of them'.

y procedemos a ejecutar la regla YARA sobre el archivo ejecutable de malware con el siguiente comando:

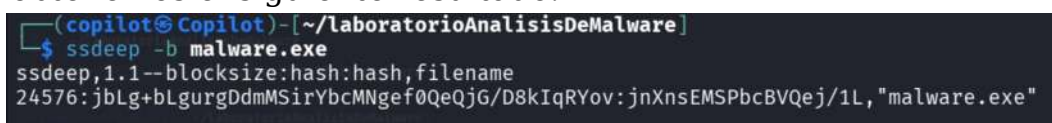
```
yara reglas_wannacry.yar malware.exe
```

Generación de hash con ssdeep

Ahora generamos el hash ssdeep del archivo ejecutable de malware con el siguiente comando:

```
ssdeep -b malware.exe
```

y obtenemos el siguiente resultado:



The screenshot shows a terminal window with the command 'ssdeep -b malware.exe' executed. The output is a long string representing the ssdeep hash: '24576:jbLg+blgurgDdmMSirYbcMNgef0QeQjG/D8kIqRYov:jnXnsEMSPbcBVQej/1L,"malware.exe"'. The terminal prompt indicates the user is in the directory '~/laboratorioAnálisisDeMalware'.

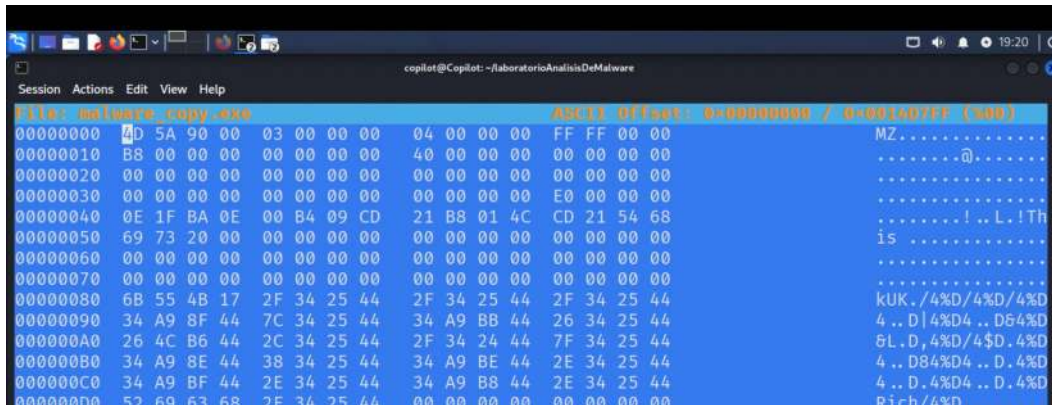
y ahora creamos una copia del malware y le hacemos un cambio en el archivo ejecutable de malware con el siguiente comando:

`cp malware.exe malware_copia.exe`

```
total 3884
-rw-r--r-- 1 copilot copilot 1366016 Jun 23 19:10 malware_copy.exe
```

Ahora modificamos el archivo ejecutable de malware con el siguiente comando:

`hexeditor malware_copia.exe`

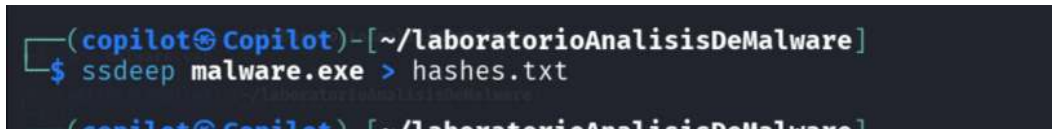


en

donde se elimino el mensaje de originalidad del malware, y luego de guardar los cambios, generamos el hash ssdeep del archivo ejecutable de malware modificado con el siguiente comando:

`ssdeep malware.exe > hashes.txt`

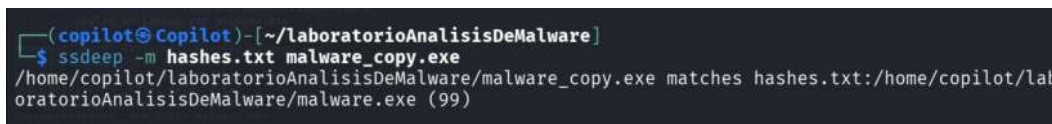
guardamos el hash del archivo original



y

realizamos la comparacion de similitud entre el hash del archivo original y el hash del archivo modificado con el siguiente comando:

`ssdeep -m hashes.txt malware_copy.exe`



Generación de hash con python-ssdeep

Creamos el código de python para generar el hash ssdeep del archivo ejecutable de malware con el siguiente contenido:

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
```

Integración YARA + SSDEEP - Laboratorio de análisis de malware

Autor: Copilot

Demuestra:

(b) Deteccion combinada de reglas YARA con similitud SSDEEP sobre muestras reales.

(c) Comparacion: YARA regular vs YARA integrada con SSDEEP.

La idea: YARA detecta por PATRON; SSDEEP detecta por SIMILITUD. Si una variante

cambia y YARA no la ve, SSDEEP puede pillarla si se parece a una muestra conocida.

Requisitos (en Kali ya instalados via apt):

sudo apt install -y python3-yara python3-ssdeep

Uso:

*python3 yara_ssdeep_integrado.py <carpeta_muestras> <reglas.yar>
<muestra_referencia>*

Ejemplo:

*python3 yara_ssdeep_integrado.py ./muestras reglas_wannacry.yar
malware.exe
"""*

```
import sys
import os
import yara
import ssdeep
```

```
# Similitud SSDEEP minima (0-100) para considerar que algo es  
"variante"
```

```
# de la muestra de referencia. Ajustable segun tus datos.
```

```
UMBRAL_SSDEEP = 70
```

```
def main():
```

```
    if len(sys.argv) != 4:  
        print(__doc__)  
        sys.exit(1)
```

```
    carpeta, reglas_path, referencia = sys.argv[1], sys.argv[2], sys.argv[3]
```

```
# Compilar las reglas YARA (deteccion por patron)
try:
    reglas = yara.compile(filepath=reglas_path)
except yara.Error as e:
    print(f"[!] Error compilando las reglas YARA: {e}")
    sys.exit(1)

# Hash SSDEEP de la muestra de referencia (la "conocida")
ref_hash = ssdeep.hash_from_file(referencia)
print(f"Muestra de referencia : {os.path.basename(referencia)}")
print(f"Hash SSDEEP referencia: {ref_hash}\n")

# Cabecera de la tabla
print(f'{'archivo':28} {'YARA':5} {'ssdeep%':8} {'detectado por'}')
print("-" * 72)

solo_yara = 0      # cuantos detecta YARA regular
integrado = 0      # cuantos detecta YARA + SSDEEP
variantes = 0      # cuantos detecta SSDEEP que YARA NO vio

for nombre in sorted(os.listdir(carpeta)):
    ruta = os.path.join(carpeta, nombre)
    if not os.path.isfile(ruta):
        continue

    # 1) DETECCION YARA (regular) -----
    matches = reglas.match(ruta)
    yara_hit = bool(matches)

    # 2) SIMILITUD SSDEEP -----
    try:
        h = ssdeep.hash_from_file(ruta)
        sim = ssdeep.compare(ref_hash, h)
    except Exception:
        sim = 0

    # 3) VEREDICTO INTEGRADO -----
    if yara_hit:
        quien = "YARA" + (f" + SSDEEP {sim}%" if sim >=
UMBRALES_SSDEEP else "")
```

```

detectado = True
elif sim >= UMBRAL_SSDEEP:
    quien = f"SOLO SSDEEP ({sim}%) <- YARA no la vio"
    detectado = True
    variantes += 1
else:
    quien = "ninguno (no relacionado)"
    detectado = False

if yara_hit:
    solo_yara += 1
if detectado:
    integrado += 1

marca = "SI" if yara_hit else "no"
print(f"{nombre:28} {marca:5} {sim:<8} {quien}")

```

```

# Resumen: YARA regular vs YARA + SSDEEP (punto c) -----
print("\n" + "=" * 72)
print("COMPARACION (punto c):")
print(f" Reglas YARA regulares detectan : {solo_yara}")
print(f" YARA integrada con SSDEEP detecta: {integrado}")
    f" (+{variantes} variante(s) que YARA no vio)")
print("=" * 72)

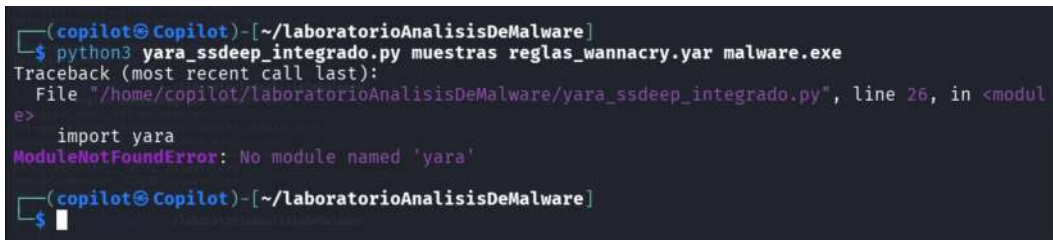
```

```

if __name__ == "__main__":
    main()

```

Encontramos el siguiente error al ejecutar el script de python, en el que indica que no encuentra el modulo Yara



para solucionarlo realizamos la instalación del modulo Yara con el siguiente comando:



y ejecutamos nuevamente el script de python, obteniendo el siguiente resultado:

```
(copilot@Copilot)-[~/LaboratorioAnálisisDeMalware]
$ python3 yara_ssdeep_integrado.py muestras reglas_wannacry.yar malware.exe
Muestra de referencia : malware.exe
Hash SSDEEP referencia: 24576:jbLg+bLgurgDdmMSirYbcMNgf0QeQjG/D8kIqRYov:jnXnsEMSPbcBVQej/1L

archivo          YARA  ssdeep%  detectado por
-----
malware.exe      SI     100      YARA + SSDEEP 100%
malware_copy.exe SI     99       YARA + SSDEEP 99%

COMPARACION (punto c):
Reglas YARA regulares detectan : 2
YARA integrada con SSDEEP detecta: 2 (+0 variante(s) que YARA no vio)
```

3. Conclusiones

SSDEEP permite comparar archivos por similitud y no únicamente por coincidencia exacta, lo que lo convierte en una herramienta útil para detectar variantes de malware o archivos relacionados que un hash tradicional (MD5/SHA256) nunca asociaría, ya que cualquier cambio mínimo altera por completo el hash criptográfico.

La integración de YARA con SSDEEP en Python combina dos enfoques complementarios —detección por patrones específicos y detección por similitud—. En el laboratorio se evidenció el valor de esta combinación: una variante con sus cadenas alteradas evade la regla YARA, pero es detectada por SSDEEP gracias a su alta similitud con la muestra original, demostrando que la detección integrada es más robusta que la detección únicamente por reglas.

Ninguna de las dos técnicas reemplaza a la otra: funcionan mejor en conjunto, especialmente en entornos de investigación. Además, el trabajo con muestras reales de malware exige medidas de seguridad estrictas (máquina virtual aislada, snapshots, configuración de red controlada y no ejecutar el binario) para evitar incidentes en el sistema anfitrión.

4. Referencias

Alvarez, V. (2024). *YARA documentation — The pattern matching swiss knife for malware researchers*. VirusTotal / YARA Project.
<https://yara.readthedocs.io>

Breitinger, F., & Baier, H. (2012). Similarity preserving hashing: Eligible properties and a new algorithm MRSH-v2. *Digital Investigation*, 9(S1), S47–S56. <https://doi.org/10.1016/j.diin.2012.05.006>

- DiMaggio, J. (2023). *The Art of Malware Analysis*. No Starch Press.
- Kornblum, J. (2006). Identifying almost identical files using context triggered piecewise hashing. *Digital Investigation*, 3(S1), 91–97.
<https://doi.org/10.1016/j.diin.2006.06.015>
- Ligh, M. H., Case, A., Levy, J., & Walters, A. (2014). *The Art of Memory Forensics: Detecting Malware and Threats in Windows, Linux, and Mac Memory*. Wiley. ISBN: 978-1118825099
- MalwareBazaar — Abuse.ch. (2024). *MalwareBazaar — Share malware samples with the community*. <https://bazaar.abuse.ch>
- NIST. (2023). *National Software Reference Library (NSRL)*. U.S. Department of Commerce.
<https://www.nist.gov/software-quality-group/national-software-reference-library-nsrl>
- Oliver, J., Cheng, C., & Chen, Y. (2013). TLSH — A locality sensitive hash. *4th Cybercrime and Trustworthy Computing Workshop*, IEEE.
<https://doi.org/10.1109/CTC.2013.9>
- Python Software Foundation. (2024). *Python 3 documentation — os module*. <https://docs.python.org/3/library/os.html>
- python-ssdeep contributors. (2023). *python-ssdeep: Python wrapper for ssdeep*. PyPI / GitHub. <https://pypi.org/project/ssdeep/>
- Roussev, V. (2010). Data fingerprinting with similarity digests. *Advances in Digital Forensics VI*, IFIP AICT 337. Springer.
https://doi.org/10.1007/978-3-642-15506-2_15
- Sikorski, M., & Honig, A. (2012). *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*. No Starch Press. ISBN: 978-1593272906
- ssdeep Project. (2023). *ssdeep — Fuzzy hashing program and library*. <https://ssdeep-project.github.io/ssdeep/>
- VirusTotal. (2024). *SSDEEP fuzzy hashing in file analysis*. VirusTotal Documentation. <https://docs.virustotal.com>
- VirusTotal. (2024). *yara-python: Python bindings for YARA*. GitHub.
<https://github.com/VirusTotal/yara-python>